Module 06 — Logging Module

Phase: Advanced Python
Previous module: File Handling
Next module: Git and GitHub

Module Outcome:

- Why logging beats `print` in real apps
- Logging levels — DEBUG → CRITICAL (and NOTSET)
- `logging.basicConfig` — file, level, format, datefmt, filemode
- Writing exceptions with `logging.exception`
- Problems with the root logger
- **Advanced:** Logger → Handler → Formatter → Filter architecture
- Named loggers, file + console handlers, different levels per handler
- Propagation, reusable `logging_config.py`, rotating / timed handlers
- Awareness — never log secrets; path to Django / FastAPI logging

---

What is Logging?

It is highly recommended to store **complete application flow** and **exception information** in a file. That process is called **logging**.

Main advantages:

1. Log files help while **debugging** (“when did it fail?”)
2. You can collect **statistics** (requests per day, error counts, …)
3. A real app runs on a server — **nobody is staring at the terminal**

Python provides the built-in module `logging`.

Where do we use in realtime?

- **Employee Management Portal** — every expense add / CSV error in `logs/app.log`
- Servers / Django / Flask — request flow and exceptions without a live console
- **Apollo Hospitals** / **HDFC Bank** — audit-style events; **never** log passwords, OTPs, or card numbers

```
print()     →  good for class / quick debug
logging     →  good for real applications
```

A clerk says: **“My expense wasn’t saved.”** You need answers:

```
When did it happen?
What operation was running?
Was it a warning, or did something fail?
```

---

1. Logging levels

Depending on the type of information, logging data is divided into **six** levels:

Level	Value	Think of it as	Expense Tracker example
CRITICAL	50	Very serious — needs high attention	Application cannot start
ERROR	40	Serious error	CSV could not be read
WARNING	30	Caution / unusual	Daily limit almost reached
INFO	20	Important normal information	Expense added
DEBUG	10	Developer detail	Reading row 25
NOTSET	0	Level not set	—

Remember the meaning, not only the numbers.

By default, while executing a Python program, only **WARNING and higher** messages are shown (if you do not configure a lower level).

A logger set to **INFO hides** DEBUG.

> **Interview:** `print` is for quick demos; `logging` gives levels, timestamps, and file/console handlers you can filter.

---

2. How to implement logging — `basicConfig`

To perform logging you typically:

1. Create / name a **log file**
2. Specify **which level** (and higher) to store

Use `logging.basicConfig(...)`.

```
import logging

logging.basicConfig(filename="log.txt", level=logging.WARNING)
```

That creates `log.txt` and stores **WARNING or higher**.

After configuration, write messages with:

```
logging.debug("...")
logging.info("...")
logging.warning("...")
logging.error("...")
logging.critical("...")
```

Program — WARNING and higher only:

```
import logging
```

```
logging.basicConfig(filename="log.txt", level=logging.WARNING)
print("Logging Demo")
logging.debug("Debug Information")
logging.info("info Information")
logging.warning("warning Information")
logging.error("error Information")
logging.critical("critical Information")
```

`log.txt`:

```
WARNING:root:warning Information
ERROR:root:error Information
CRITICAL:root:critical Information
```

If you set `level=logging.DEBUG`, **all** messages are written.

```
import logging

logging.basicConfig(filename="log.txt", level=logging.DEBUG)
print("Logging Demo")
logging.debug("Debug Information")
logging.info("info Information")
logging.warning("warning Information")
logging.error("error Information")
logging.critical("critical Information")
```

`log.txt`:

```
DEBUG:root:Debug Information
INFO:root:info Information
WARNING:root:warning Information
ERROR:root:error Information
CRITICAL:root:critical Information
```

Note:

- If you omit `level`, the default is **WARNING (30)**.
- If you omit `filename`, messages go to the **console**.
- `basicConfig` runs **once** per process. Restart the interpreter if a second call seems to do nothing.
- Configure logging in `main`, not at import time inside a library.

---

3. Append vs overwrite — `filemode`

By default, data is **appended** to the log file. To overwrite previous data, use `filemode='w'`.

```
# append (default)
logging.basicConfig(filename="log786.txt", level=logging.WARNING)
logging.basicConfig(filename="log786.txt", level=logging.WARNING, filemode="a")

# overwrite previous data
logging.basicConfig(filename="log786.txt", level=logging.WARNING, filemode="w")
```

---

4. Formatting log messages

Use the `format` keyword.

Level name only:

```
logging.basicConfig(format="%(levelname)s")
```

```
WARNING
ERROR
CRITICAL
```

Level + message:

```
logging.basicConfig(format="%(levelname)s:%(message)s")
```

```
WARNING:warning Information
ERROR:error Information
CRITICAL:critical Information
```

Add timestamp:

```
logging.basicConfig(format="%(asctime)s:%(levelname)s:%(message)s")
```

```
2018-06-15 11:50:08,325:WARNING:warning Information
2018-06-15 11:50:08,372:ERROR:error Information
2018-06-15 11:50:08,372:CRITICAL:critical Information
```

Change date/time format — `datefmt`:

```
logging.basicConfig(
    format="%(asctime)s:%(levelname)s:%(message)s",
    datefmt="%d/%m/%Y %I:%M:%S %p",
)
```

```
15/06/2018 12:04:31 PM:WARNING:warning Information
```

Token	Meaning
`%I`	12-hour clock
`%H`	24-hour clock

Docs: LogRecord attributes · strftime

Class-friendly expense style:

```
import logging

logging.basicConfig(level=logging.INFO, format="%(levelname)s %(message)s")
```

```
logging.debug("Reading row 25 — this will NOT appear at INFO level")
logging.info("Expense added: ₹120 Tea")
logging.warning("Daily limit almost reached")
logging.error("Failed to parse amount: 'abc'")
```

Expected Output:

```
INFO Expense added: ₹120 Tea
WARNING Daily limit almost reached
ERROR Failed to parse amount: 'abc'
```

---

5. Writing exceptions to the log — `logging.exception`

Use `logging.exception(msg)` inside an `except` block. It logs at **ERROR** and includes the **traceback**.

```
import logging

logging.basicConfig(
    filename="mylog.txt",
    level=logging.INFO,
    format="%(asctime)s:%(levelname)s:%(message)s",
    datefmt="%d/%m/%Y %I:%M:%S %p",
)

logging.info("A new Request Came")
try:
    x = int(input("Enter First Number:"))
    y = int(input("Enter Second Number:"))
    print("The Result:", x / y)
except ZeroDivisionError as msg:
    print("cannot divide with zero")
    logging.exception(msg)
except ValueError as msg:
    print("Please provide int values only")
    logging.exception(msg)
logging.info("Request Processing Completed")
```

Successful run prints the result; bad input prints a clear message. `mylog.txt` keeps INFO lines plus ERROR + traceback for failures.

Remember: Call `logging.exception` only inside an exception handler (so a traceback exists).

---

6. Problems with the root logger

If you do not define your own logger, the **root** logger is used.

Once you call `basicConfig` on the root logger, that configuration is **fixed** for the process — a later `basicConfig` usually does nothing.

Demo:

`student.py`:

```
import logging
```

```
logging.basicConfig(filename="student.log", level=logging.INFO)
logging.info("info message from student module")
```

`test.py`:

```
import logging
import student

logging.basicConfig(filename="test.log", level=logging.DEBUG)
logging.debug("debug message from test module")
```

Only `student.log` gets the INFO line. The `basicConfig` in `test.py` does **not** take effect — root was already configured in `student`.

Problems with root / `basicConfig` alone:

1. Configuration is final once set — hard to change later
2. Often one destination at a time (console **or** file), not both easily
3. Hard to use different configs at different levels / modules
4. Cannot cleanly give each module its own log file

Solution: create your **own** customized loggers — the rest of this module.

Advanced Logging — Creating Custom Loggers

Basic `logging.info(...)` is enough for a tiny script. A Python Full Stack app has many parts — you need **Logger**, **Handler**, **Formatter**, and (optionally) **Filter**.

7. WHY — Why create a custom logger?

In a small program, this is enough:

```
import logging

logging.info("User logged in")
```

Real applications have different parts:

```
Application
│
├── User Service
├── Payment Service
├── Order Service
└── Database Service
```

You may want **separate logging behaviour** for each part — for example:

```
app.log
payment.log
database.log
```

That is where **custom loggers** are useful.

---

8. UNDERSTAND — The logging architecture

Python logging is built around four important components:

```
Logger
   ↓
Handler
   ↓
Formatter
   ↓
Output
```

Component	Job	Example
Logger	Creates the log record	`logger.info("User logged in")`
Handler	Decides **where** the log goes	Console (`StreamHandler`), file (`FileHandler`)
Formatter	Decides **how** the log looks	`2026-09-10 23:30:10 - INFO - User logged in`
Filter	Controls which records are allowed through	Drop DEBUG from one handler; keep secrets out

Interview line to remember: *A Logger creates log records, Handlers determine where they go, and Formatters determine how those records are displayed.*

---

9. SEE — Creating a custom logger

Instead of the root logger (`logging.info("Hello")`), create your own:

```
import logging

logger = logging.getLogger("my_app")
logger.setLevel(logging.DEBUG)

logger.info("Application started")
logger.warning("Something unusual happened")
```

`logging.getLogger("my_app")` creates or **retrieves** a logger named `my_app`.

> **Note:** Calling `getLogger` with the same name twice returns the **same** logger object — that is how modules share configuration.

---

10. Why logger names matter

In a larger application:

```
user_logger = logging.getLogger("app.user")
payment_logger = logging.getLogger("app.payment")
database_logger = logging.getLogger("app.database")
```

Now you can see **where** a message came from:

```
app.user
app.payment
app.database
```

That is very useful when debugging large applications (and later Django / FastAPI services).

Use a class name when it helps:

```
logger = logging.getLogger(PaymentService.__name__)
```

---

11. Creating a File Handler

A logger itself does **not** decide where the message goes. A **handler** does.

```
import logging

logger = logging.getLogger("app")
logger.setLevel(logging.DEBUG)

file_handler = logging.FileHandler("app.log", encoding="utf-8")
logger.addHandler(file_handler)

logger.info("Application started")
logger.error("Something went wrong")
```

Now logs are written to `app.log`.

Note: Logger level is the first gate. Handler levels must be **supported** by the logger level — set the logger to the **lowest** level you care about; handlers filter further.

Logger	Handler	Result
DEBUG	INFO	Valid — INFO and higher reach the handler
INFO	DEBUG	DEBUG never reaches the handler

---

12. Adding a Formatter

Without a formatter, logs are not very informative. Create one and attach it to the handler:

```
import logging

logger = logging.getLogger("app")
logger.setLevel(logging.DEBUG)

file_handler = logging.FileHandler("app.log", encoding="utf-8")
formatter = logging.Formatter(
    "%(asctime)s - %(name)s - %(levelname)s - %(message)s"
)
file_handler.setFormatter(formatter)
logger.addHandler(file_handler)

logger.info("Application started")
```

```
logger.warning("Low disk space")
logger.error("Database connection failed")
```

Example `app.log`:

```
2026-09-10 23:40:10,123 - app - INFO - Application started
2026-09-10 23:40:10,125 - app - WARNING - Low disk space
2026-09-10 23:40:10,126 - app - ERROR - Database connection failed
```

---

13. Logger + Handler + Formatter (relationship)

Students should remember this chain:

```
Logger
                  |
             creates record
                  ↓
               Handler
                  |
          decides destination
                  ↓
              Formatter
                  |
          decides appearance
                  ↓
             Log Output
```

Concrete path:

```
Logger
  ↓
FileHandler
  ↓
Formatter
  ↓
app.log
```

---

14. Logging to console AND file

Real applications often need both:

```
Logger
              /    \
             /      \
            ↓        ↓
       Console     File
       Handler    Handler
```

```
import logging
import os

os.makedirs("logs", exist_ok=True)

logger = logging.getLogger("app")
logger.setLevel(logging.DEBUG)
```

```
logger.handlers.clear()  # avoid duplicate lines if re-run in the same process

console_handler = logging.StreamHandler()
file_handler = logging.FileHandler("logs/app.log", encoding="utf-8")

formatter = logging.Formatter(
    "%(asctime)s - %(levelname)s - %(message)s"
)
console_handler.setFormatter(formatter)
file_handler.setFormatter(formatter)

logger.addHandler(console_handler)
logger.addHandler(file_handler)

logger.info("Application started")
```

The same message can appear on the **console** and in `logs/app.log`.

---

15. Different levels for different handlers

This is a powerful, production-style idea.

Suppose you want:

- **Console** → WARNING and above
- **File** → DEBUG and above

```
import logging
import os

os.makedirs("logs", exist_ok=True)

logger = logging.getLogger("app")
logger.setLevel(logging.DEBUG)
logger.handlers.clear()

console_handler = logging.StreamHandler()
console_handler.setLevel(logging.WARNING)

file_handler = logging.FileHandler("logs/app.log", encoding="utf-8")
file_handler.setLevel(logging.DEBUG)

formatter = logging.Formatter(
    "%(asctime)s - %(levelname)s - %(message)s"
)
console_handler.setFormatter(formatter)
file_handler.setFormatter(formatter)

logger.addHandler(console_handler)
logger.addHandler(file_handler)

logger.debug("Debug information")
logger.info("Application started")
logger.warning("Low disk space")
logger.error("Database error")
```

Console:

```
WARNING - Low disk space
```

```
ERROR - Database error
```

`logs/app.log`:

```
DEBUG - Debug information
INFO - Application started
WARNING - Low disk space
ERROR - Database error
```

ATM / expense style (same idea): INFO+ to file, WARNING+ to console.

---

16. Preventing duplicate logs — propagation

When working with custom loggers, students may accidentally see the **same message multiple times**. That is often **propagation**.

Python loggers form a hierarchy:

```
app
│
├── app.user
├── app.payment
└── app.database
```

Child loggers can **propagate** messages up to their parent (and eventually root). Disable it when a child has its own handlers:

```
logger = logging.getLogger("app.payment")
logger.propagate = False
```

> **Remember:** If you see doubles, check (1) `logger.handlers` added twice, and (2) `propagate` still `True` while a parent also has handlers.

---

17. Reusable logging configuration — `logging_config.py`

Instead of configuring logging in every Python file, centralise it:

`logging_config.py`:

```
import logging
import os


def get_logger(name):
    logger = logging.getLogger(name)
    logger.setLevel(logging.DEBUG)

    if not logger.handlers:  # attach once — avoid duplicates on re-import
        os.makedirs("logs", exist_ok=True)
        file_handler = logging.FileHandler("logs/app.log", encoding="utf-8")
        formatter = logging.Formatter(
            "%(asctime)s - %(name)s - %(levelname)s - %(message)s"
        )
        file_handler.setFormatter(formatter)
        logger.addHandler(file_handler)
```

```
        logger.propagate = False

    return logger
```

Another file:

```
from logging_config import get_logger

logger = get_logger("user_service")
logger.info("User created")
```

Payment module:

```
from logging_config import get_logger

logger = get_logger("payment_service")
logger.info("Payment started")
```

`logs/app.log` idea:

```
... - user_service - INFO - User created
... - payment_service - INFO - Payment started
```

This is the habit you want before Django / FastAPI — **one place** owns logging setup.

---

18. Rotating log files

What if `app.log` grows for months? It can become huge. Use a **rotating** handler:

```
import logging
from logging.handlers import RotatingFileHandler

logger = logging.getLogger("app")
logger.setLevel(logging.INFO)

handler = RotatingFileHandler(
    "app.log",
    maxBytes=1_000_000,
    backupCount=3,
    encoding="utf-8",
)
formatter = logging.Formatter(
    "%(asctime)s - %(levelname)s - %(message)s"
)
handler.setFormatter(formatter)
logger.addHandler(handler)

logger.info("Application started")
```

Conceptually:

```
app.log
app.log.1
```

```
app.log.2
app.log.3
```

When the main file reaches `maxBytes`, older logs rotate; only `backupCount` backups are kept.

---

19. Timed rotating logs

Sometimes apps rotate by **time**, not size:

```
from logging.handlers import TimedRotatingFileHandler

handler = TimedRotatingFileHandler(
    "app.log",
    when="midnight",
    backupCount=7,
    encoding="utf-8",
)
```

That can produce daily log files while retaining a limited number of old files.

---

20. Final advanced example (type with the class)

One example that ties it together:

```
import logging
import os
from logging.handlers import RotatingFileHandler

os.makedirs("logs", exist_ok=True)

logger = logging.getLogger("student_app")
logger.setLevel(logging.DEBUG)
logger.handlers.clear()
logger.propagate = False

console_handler = logging.StreamHandler()
console_handler.setLevel(logging.WARNING)

file_handler = RotatingFileHandler(
    "logs/student_app.log",
    maxBytes=1_000_000,
    backupCount=3,
    encoding="utf-8",
)
file_handler.setLevel(logging.DEBUG)

formatter = logging.Formatter(
    "%(asctime)s - %(name)s - %(levelname)s - %(message)s"
)
console_handler.setFormatter(formatter)
file_handler.setFormatter(formatter)

logger.addHandler(console_handler)
logger.addHandler(file_handler)

logger.debug("Debug information")
logger.info("Student application started")
```

```
logger.warning("Student not found")
logger.error("Database operation failed")
```

This single example demonstrates:

- custom logger
- logging levels
- handlers (console + file)
- formatter
- rotating logs
- different levels per handler

---

21. Awareness — config file and privacy

Hard-coding handlers is fine for class. Larger apps may also load `logging.config.fileConfig("logging.conf")` or dictConfig — same ideas (loggers / handlers / formatters), different packaging.

Never log passwords, OTPs, full card numbers, Aadhaar, or session tokens (GDPR / India DPDP). Log that an auth attempt failed, not the secret.

Step-by-Step: wire logging into the expense slice (basic → custom)

You already have CSV + JSON from File Handling. Start with `basicConfig`, then prefer a named logger for the lab.

A — Quick trail with `basicConfig`:

```
import csv
import json
import logging
import os

os.makedirs("data", exist_ok=True)
os.makedirs("logs", exist_ok=True)

logging.basicConfig(
    filename="logs/app.log",
    level=logging.INFO,
    format="%(asctime)s %(levelname)s %(message)s",
    datefmt="%d/%m/%Y %H:%M:%S",
)

config = {
    "app_name": "ExpenseTracker",
    "currency": "INR",
    "max_daily_limit": 5000,
    "categories": ["food", "travel", "rent"],
}
with open("data/config.json", "w", encoding="utf-8") as f:
    json.dump(config, f, indent=2)

rows = [
    {"date": "2026-09-04", "category": "food", "amount": "120", "note": "Tea"},
    {"date": "2026-09-04", "category": "travel", "amount": "50", "note": "Metro"},
]
with open("data/expenses.csv", "w", newline="", encoding="utf-8") as f:
    writer = csv.DictWriter(f, fieldnames=["date", "category", "amount", "note"])
```

```
    writer.writeheader()
    writer.writerows(rows)
logging.info("Wrote data/expenses.csv")

total = 0
with open("data/expenses.csv", newline="", encoding="utf-8") as f:
    for row in csv.DictReader(f):
        print(f"{row['note']}: ₹{row['amount']}")
        total += int(row["amount"])

with open("data/config.json", "r", encoding="utf-8") as f:
    limit = json.load(f)["max_daily_limit"]

logging.info("Day total ₹%s (limit ₹%s)", total, limit)
print("Day total ₹%s (limit ₹%s)" % (total, limit))
print("Check logs/app.log")
```

Expected Output (console):

```
Tea: ₹120
Metro: ₹50
Day total ₹170 (limit ₹5000)
Check logs/app.log
```

B — Same product, custom logger (stretch in class): replace `basicConfig` with `getLogger("expense")` + file handler + formatter (sections 11–14). Console WARNING+ / file DEBUG+ if time allows (section 15).

Then open the kit: 01_python_expense_tracker — add logging to the expense CLI. Type it — do not paste `solution/`.

Practice Exercises

1. Configure `basicConfig` at DEBUG to `demo.log`. Log one message at each level. Open the file — how many lines?

Model answer: Five lines (DEBUG through CRITICAL).

2. Repeat with `level=logging.WARNING`. Which messages disappear?

Model answer: DEBUG and INFO are filtered out.

3. Format messages as `%(asctime)s | %(levelname)s | %(message)s` with `datefmt="%Y-%m-%d %H:%M:%S"`.
4. Wrap `int(input(...))` in `try` / `except ValueError` and call `logging.exception` on failure. Confirm the traceback is in the log file.
5. Build logger `atm`: INFO+ → `atm.log`, WARNING+ → console. Log one INFO and one WARNING.

Model answer:

```
import logging

logger = logging.getLogger("atm")
logger.setLevel(logging.INFO)
logger.handlers.clear()
logger.propagate = False
fmt = logging.Formatter("%(levelname)s %(message)s")
fh = logging.FileHandler("atm.log", encoding="utf-8")
fh.setLevel(logging.INFO)
```

```
ch = logging.StreamHandler()
ch.setLevel(logging.WARNING)
fh.setFormatter(fmt)
ch.setFormatter(fmt)
logger.addHandler(fh)
logger.addHandler(ch)
logger.info("ATM session started")
logger.warning("Cash drawer low")
```

6. Write `logging_config.py` with `get_logger(name)`. From two modules, log as `user_service` and `payment_service` into the same file. Confirm `%(name)s` differs on each line.
7. Switch the file handler to `RotatingFileHandler(maxBytes=1000, backupCount=2)` and write in a loop until a backup file appears.

Mini-Project (Assignment)

A — DailyExpenseLogger (core lab)

Title: DailyExpenseLogger
Customer: Employee Management Portal finance — same CSV/JSON tracker as File Handling; every operation leaves a trail in `logs/app.log`.
Lab kit: 15_Lab_Assets/01_python_expense_tracker/

Piece	Detail
`logs/app.log`	Prefer a **named** custom logger
Levels	INFO for success; ERROR for failures
Resilience	Bad/missing files → log + clear msg

Done when:

- [ ] Logging writes to a log file (not only `print`)
- [ ] At least one `logging.exception` (or ERROR) path for bad input / missing file
- [ ] No passwords / secrets in logs
- [ ] Pushed to GitHub

B — Advanced: Student Management System logger

```
student_app/
│
├── main.py
├── student_service.py
├── logging_config.py
└── logs/
    └── student_app.log
```

Requirements:

- `logging_config.py` — reusable `get_logger` (console WARNING+, file DEBUG+, rotating file handler)
- `student_service.py` — `add_student()`, `find_student()`, `delete_student()` each with appropriate logs
- INFO → successful operations

- WARNING → student not found
- ERROR → unexpected failures
- DEBUG → useful development detail
- `main.py` — call the three operations so students can verify console vs file behaviour

Done when:

- [ ] Central `logging_config.py` used by service + main
- [ ] Console shows WARNING+ only; file has DEBUG+
- [ ] Rotating handler configured
- [ ] Named loggers identify the service (`%(name)s`)

What you should know after this module

You should understand:

```
logging
   ↓
Logger
   ↓
Handler
   ↓
Formatter
   ↓
Output
```

and be comfortable explaining:

> A Logger creates log records, Handlers determine where they go, and Formatters determine how those records are displayed.

That is the advanced logging foundation expected before frameworks such as **Django** or **FastAPI**.

Interview Q&A

Q1. Purpose of the logging module vs `print`?
A: Logging provides levels, timestamps, handlers (file/console), configurable format, and production-ready filtering. `print` is ad-hoc debugging only. On a server, nobody is watching the terminal.

Q2. What are the standard logging levels (high → low)?
A: CRITICAL (50), ERROR (40), WARNING (30), INFO (20), DEBUG (10), plus NOTSET (0). Default threshold is WARNING if unset.

Q3. What does `basicConfig` do? Why does a second call often fail?
A: It configures the root logger (file, level, format, …). Once the root logger has handlers, later `basicConfig` calls are ignored.

Q4. Why prefer a custom logger (`getLogger`) over only root?
A: You can attach multiple handlers, set per-module names, and avoid the “config is frozen after first import” trap of root `basicConfig`.

Q5. Logger vs Handler vs Formatter?

A: Logger creates the record; Handler chooses destination (console/file); Formatter shapes the text. Optional Filter decides which records pass.

Q6. Why do logger names like `app.payment` matter?

A: They form a hierarchy and show which part of the system produced a line — essential when many modules share one log file.

Q7. `logging.error` vs `logging.exception`?

A: Both are ERROR level. `exception` also records the current exception traceback — use it inside `except`.

Q8. What is `propagate`? When set it to `False`?

A: Child loggers forward records to parents by default. Set `propagate = False` when the child has its own handlers so messages are not duplicated up the tree / root.

Q9. File handler DEBUG, console handler WARNING — what appears where?

A: DEBUG and INFO go to the file only (if logger allows them). WARNING and above go to both.

Q10. What sensitive data should never go in log files?

A: Passwords, OTPs, full card numbers, Aadhaar, session tokens — GDPR / India DPDP. Log that an auth attempt failed, not the secret itself.

Q11. How would you rotate log files when they grow too large?

A: Use `logging.handlers.RotatingFileHandler` (size) or `TimedRotatingFileHandler` (time) so old logs are renamed and a fresh file starts.

Q12. Why put setup in `logging_config.py`?

A: Centralised configuration — every module calls `get_logger("…")` once; handlers are not re-added on every import; consistent format across the app.
